



Universidad
Tecnológica
del Perú

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Facultad de Ingeniería de Sistemas y Computación

DOCUMENTACIÓN DEL PROYECTO

"Animal Shelter" – Sistema de Gestión de un Refugio de Gatos

Curso: Algoritmos y Estructuras de Datos

Profesor:

Mg. Ing. Juan Ramirez Ticona

Alumnos:

Lopez Calderon Andrea del Rosario

Guardia Quispe Luis Bryan

Arequipa, Perú – 2026

Contenido

1. Introducción	3
1.1 Objetivos	3
2. Descripción General del Sistema.....	3
3. Estructuras de Datos Implementadas	4
3.1 Nodo genérico (Nodo<T>).....	5
3.2 Pila (Stack) — Pila.java	5
3.3 Cola (Queue) — Cola.java	6
3.4 Lista Enlazada Simple — ListaEnlazadaSimple.java	6
3.5 Lista Doblemente Enlazada — ListaDobleEnlazada<T>.java.....	7
3.6 Lista Circular — ListaCircular.java.....	7
3.7 Árbol Binario de Búsqueda — ArbolBinario.java	8
3.8 Resumen de justificación (para la sustentación oral)	9
3.9 Complejidad de las operaciones principales	9
4. Diagrama UML.....	10
5. Conclusiones.....	10

1. Introducción

El presente documento describe el proyecto "Animal Shelter", una aplicación de escritorio desarrollada en Java cuyo propósito es gestionar la información de un refugio de gatos: administradores, voluntarios, adoptantes y los propios gatos disponibles para adopción. El sistema permite registrar, buscar, modificar y eliminar información de cada uno de estos actores, además de manejar la persistencia de los datos mediante archivos de texto.

Más allá de su funcionalidad como sistema de gestión, el valor académico principal de este proyecto para el curso de Algoritmos y Estructuras de Datos radica en que todas las colecciones de información NO se almacenan usando las clases predefinidas de Java (como ArrayList, HashMap o LinkedList), sino que se implementan de forma manual las estructuras de datos clásicas: pilas, colas, listas enlazadas simples, listas doblemente enlazadas, listas circulares y árboles binarios de búsqueda.

De esta manera, cada entidad del sistema (Administradores, Voluntarios, Adoptantes y Gatos) queda asociada a una estructura de datos distinta, lo que permite evidenciar de forma práctica el funcionamiento interno, las ventajas y las limitaciones de cada una de ellas dentro de un contexto de aplicación real.

1.1 Objetivos

- Aplicar de forma práctica los conceptos de estructuras de datos lineales (pila, cola, listas) y no lineales (árbol binario) vistos en el curso.
- Implementar dichas estructuras de manera manual (sin usar las clases del Java Collections Framework) para reforzar la comprensión de su funcionamiento interno.
- Diseñar un sistema modular, separando la lógica de negocio (Gestores), las entidades del dominio (Entidades) y las estructuras de datos (EstructurasDeDatos).
- Persistir la información gestionada por cada estructura en archivos de texto, permitiendo que los datos se mantengan entre ejecuciones del programa.

2. Descripción General del Sistema

El proyecto está organizado en paquetes que separan claramente las responsabilidades, siguiendo principios de diseño orientado a objetos:

- Entidades: contiene las clases del dominio — Persona (clase base), Administradores, Voluntarios, Adoptantes y Gatos.
- EstructurasDeDatos: contiene las implementaciones manuales de Nodo, Pila, Cola, ListaEnlazadaSimple, ListaDobleEnlazada, ListaCircular y ArbolBinario.
- Gestores: contiene la lógica de negocio (Gestor_Gatos, Gestor_Adoptante, Gestor_Voluntario, Gestor_usuarios), encargada de cargar/guardar los datos en archivos de texto y de operar sobre la estructura de datos correspondiente.
- Menus / Scanner: contienen la interfaz de consola (menús de acciones) y la clase encargada de la lectura de datos ingresados por el usuario.

Un detalle relevante del diseño es que la clase `Nodo<T>` es genérica y reutilizable: en lugar de crear una clase de nodo distinta para cada estructura, todas (Pila, Cola, Árbol Binario, Lista

Enlazada Simple, Lista Circular y Lista Doble Enlazada) comparten la misma clase Nodo, usando únicamente los atributos que cada una necesita (dato, siguiente, anterior, izquierdo, derecho o hijos).

3. Estructuras de Datos Implementadas

A continuación, se detalla cada una de las estructuras de datos implementadas manualmente en el proyecto, indicando su propósito, la entidad que administra y sus principales operaciones. La siguiente tabla resume el panorama general:

Estructura	Clase Java	Entidad administrada	Uso principal en el sistema
Pila (Stack)	Pila.java	Administradores	Registro de administradores; versión dinámica (con Nodo) y versión estática (arreglo), con push/peek.
Cola (Queue)	Cola.java	Administradores	Gestión de administradores en orden de llegada; versión estática circular y versión dinámica enlazada.
Lista Enlazada Simple	ListaEnlazadaSimple.java	Adoptantes	Reemplaza un HashMap; inserción al final u ordenada, búsqueda por DNI/nombre y eliminación.
Lista Doblemente Enlazada	ListaDobleEnlazada.java	Voluntarios (sincronizada)	Reemplaza java.util.LinkedList; permite recorrido en ambos sentidos e inserción/eliminación O(1) en los extremos.
Lista Circular	ListaCircular.java	Voluntarios	Estructura principal de voluntarios; el último nodo enlaza con el primero para simular rotación de turnos.

Estructura	Clase Java	Entidad administrada	Uso principal en el sistema
Árbol Binario de Búsqueda	ArbolBinario.java	Gatos	Inserción y búsqueda por ID en $O(\log n)$ aproximado; búsqueda por nombre y recorrido en preorden.
Nodo genérico	Nodo.java	—	Clase reutilizada por todas las estructuras anteriores (dato, siguiente, anterior, izquierdo, derecho, hijos).

Para cada estructura se explica: (a) el problema real del sistema que resuelve, (b) por qué esa estructura es la más adecuada frente a otras alternativas, y (c) cómo está aplicada exactamente en el código (clases Gestor que la usan y métodos concretos), de modo que sirva como sustento para explicar y defender las decisiones de diseño.

3.1 Nodo genérico (Nodo<T>)

Es la unidad base reutilizada por todas las demás estructuras. Contiene los atributos dato, siguiente y anterior (para listas), izquierdo y derecho (para el árbol binario) y una lista hijos (pensada para árboles n-arios).

Justificación: en vez de crear seis clases de nodo distintas (una por estructura), se optó por una única clase `Nodo<T>` genérica que reutiliza los mismos campos según la estructura que la use. Esto reduce la duplicación de código y refleja un principio de diseño (reutilización/DRY) que también se explica fácilmente ante el profesor: cada estructura simplemente ignora los atributos que no necesita.

3.2 Pila (Stack) — Pila.java

Aplicación concreta en el proyecto: la clase `Gestor_usuarios` utiliza una Pila para llevar el historial de inicios de sesión (logins) exitosos de los administradores. Declara dos instancias: `historialLogins` (pila dinámica) e `historialLoginsEstatico` (pila estática de capacidad 2), y el usuario elige en tiempo de ejecución qué variante usar mediante la variable `tipoPila`.

Por qué una Pila y no otra estructura: el requisito de negocio es "consultar rápidamente cuál fue el último login realizado". Una pila sigue el principio LIFO (Last In, First Out — el último en entrar es el primero en salir), por lo que el administrador que inició sesión más recientemente queda siempre en el tope. Esto permite obtener ese dato en tiempo $O(1)$ usando `peek()`, sin recorrer

ninguna lista. Usar una lista simple habría obligado a recorrerla entera para llegar al último elemento insertado.

- Pila dinámica: enlazada mediante `Nodo<Administradores>`. El método `push(elemento)` crea un nuevo nodo, lo enlaza como nuevo tope (`nodo.siguiente = tope; tope = nodo`) y crece sin límite de tamaño — $O(1)$.
- Pila estática: basada en un arreglo (`pilaEstatica`) de capacidad fija definida en el constructor `Pila(int capacidad)`. `pushEstatico()` valida overflow antes de insertar (`topeEstatico == capacidadEstatica - 1`) y lanza una excepción si la pila está llena, evidenciando la limitación clásica de una estructura estática frente a la dinámica.
- En ambos casos, `guardarHistorial()` llama a `peek()` / `peekEstatico()` para escribir en un archivo (`HistorialLogins.txt`) exclusivamente el último login registrado, que es justamente la operación que la pila resuelve de forma natural y eficiente.

3.3 Cola (Queue) — Cola.java

Aplicación concreta en el proyecto: la misma clase `Gestor_usuarios` usa una Cola por cada administrador (`Map<Administradores, Cola>`) para registrar sus intentos fallidos de inicio de sesión consecutivos, en el método `IntentosLoginFallidos()`.

Por qué una Cola y no otra estructura: el requisito de negocio es "contar intentos fallidos en el orden en que ocurrieron y bloquear la cuenta al llegar a un límite". Una cola sigue el principio FIFO (First In, First Out — el primero en entrar es el primero en salir), que es exactamente el orden cronológico en que deben contabilizarse los intentos. Cada intento fallido se agrega con `enqueue()/enqueueDinamico()`; cuando el tamaño de la cola alcanza el límite (2 intentos en el código), la cuenta se bloquea automáticamente (`valor.setBloqueado(true)`) y se registra una hora de desbloqueo. Al iniciar sesión correctamente, o al pasar el tiempo de bloqueo, la cola se vacía con `clear()`, reiniciando el conteo.

- Cola estática circular: usa un arreglo de tamaño fijo (`datos`) y los índices `frente/fin`, que "dan la vuelta" al llegar al final del arreglo (`fin = 0` si `fin == capacidad`), reutilizando así los espacios liberados en vez de desperdiciar memoria.
- Cola dinámica: enlazada mediante nodos, con punteros `frenteDinamico` y `finDinamico`. `enqueueDinamico()` agrega en $O(1)$ al final de la cola sin límite de tamaño.
- Al igual que con la Pila, el usuario elige la variante (estática o dinámica) mediante la variable `tipoCola`, lo que permite comparar en la sustentación ambas implementaciones sobre el mismo caso de uso real.

3.4 Lista Enlazada Simple — ListaEnlazadaSimple.java

Aplicación concreta en el proyecto: la clase `Gestor_Adoptante` mantiene una `ListaEnlazadaSimple` (`listaAdoptantes`) como estructura principal para registrar, buscar, mostrar y eliminar a los Adoptantes del refugio.

Por qué una Lista Enlazada Simple: los adoptantes no requieren operaciones especiales de orden inverso ni de rotación, solo un registro secuencial simple con inserción ordenada y búsqueda. Por eso se eligió la estructura enlazada más básica, en reemplazo intencional de un `HashMap`, precisamente para demostrar en el curso cómo se logra manualmente (sin las colecciones

nativas de Java) una funcionalidad equivalente: cada nodo conoce únicamente al nodo siguiente, y el último apunta a null.

- `insertarOrdenado()`: es el método realmente usado por `registrar()`; inserta al adoptante comparando nombres para mantener la lista siempre ordenada alfabéticamente, sin necesidad de un paso de ordenamiento posterior — $O(n)$.
- `buscarPorDNI()` / `buscarPorNombre()`: usados por `retornarElemento()` para localizar a un adoptante ya sea por su documento o por su nombre; ambos recorren la lista secuencialmente — $O(n)$.
- `eliminar(dni)`: usado por el método `eliminar()` del gestor; localiza el nodo anterior al que se desea eliminar y reenlaza los punteros (`actual.siguiente = actual.siguiente.siguiente`), liberando el nodo intermedio.
- `mostrar()`: recorre toda la lista desde la cabeza hasta que `actual == null`, evidenciando visualmente en consola el final característico de esta estructura.

3.5 Lista Doblemente Enlazada — `ListaDobleEnlazada<T>.java`

Aplicación concreta en el proyecto: la clase `Gestor_Voluntario` mantiene una `ListaDobleEnlazada<Voluntarios>` (`listaDoble`) que se sincroniza en paralelo con la Lista Circular, cada vez que se registra, elimina o rota un voluntario.

Por qué una Lista Doblemente Enlazada: se necesitaba una segunda forma de recorrer y eliminar voluntarios aprovechando una ventaja que la lista circular simple no ofrece de forma directa: la navegación en ambos sentidos. Cada nodo de esta lista conoce tanto al nodo siguiente como al anterior, lo que permite: (1) recorrer la colección desde el final hacia el inicio sin necesidad de invertirla, y (2) eliminar un nodo en $O(1)$ si ya se tiene la referencia al objeto, sin recorrer la lista desde la cabeza para encontrar a su antecesor.

- `mostrarDobleDesdeFinal()`: recorre la lista en sentido inverso (for $i = \text{size}() - 1$ hasta 0), mostrando explícitamente que, a diferencia de la lista simple, esta estructura puede leerse en ambas direcciones.
- `remove(voluntario)`: utilizado dentro de `eliminar()` del gestor; al eliminar un voluntario se actualiza tanto la lista circular como esta lista doble, comprobando ambas operaciones en conjunto (`eliminadoCircular && eliminadoDoble`).
- Justifica en el código el reemplazo intencional de `java.util.LinkedList`: se implementa manualmente la misma idea (nodo con enlace doble) para el curso, en lugar de usar la clase nativa de Java.

3.6 Lista Circular — `ListaCircular.java`

Aplicación concreta en el proyecto: es la estructura principal (no secundaria) de `Gestor_Voluntario` para registrar, buscar y mostrar a los Voluntarios, y además implementa la funcionalidad de rotación de turnos mediante el método `rotarVoluntarios()`.

Por qué una Lista Circular: el sistema necesita simular una rotación cíclica de turnos de atención entre los voluntarios (después del último voluntario, se debe volver a asignar turno al primero). Esta es precisamente la propiedad distintiva de una lista circular: el último nodo no apunta a null sino que vuelve a apuntar a la cabeza, formando un ciclo continuo. Una lista simple o doble no

podría representar este comportamiento cíclico sin lógica adicional para "volver al inicio" manualmente.

- `insertarOrdenado()`: inserta al voluntario manteniendo el orden alfabético, igual que en la lista simple, pero conservando el enlace circular entre el último nodo y la cabeza.
- `rotar()`: el método `rotarVoluntarios()` del gestor la invoca para "correr" la cabeza de la lista al siguiente voluntario, de modo que, cuando se muestre el listado, el orden de atención cambie — esto simula asignar el turno inicial a otro voluntario en cada rotación.
- `buscarPorDNI()` / `buscarPorNombre()` / `eliminar()`: equivalentes conceptualmente a los de la lista simple, pero recorriendo la estructura hasta volver al nodo de partida (cabeza) en vez de detenerse en null.

3.7 Árbol Binario de Búsqueda — `ArbolBinario.java`

Aplicación concreta en el proyecto: la clase `Gestor_Gatos` mantiene un `ArbolBinario` donde cada Gato se inserta usando su ID como criterio de orden. El método `buscar()` del gestor delega directamente en `getArbol().buscarPorID()` y `getArbol().buscarPorNombre()`, y `mostrar()` delega en `getArbol().imprimirPreorden()`.

Por qué un Árbol Binario de Búsqueda: a diferencia de las demás entidades, los Gatos se identifican con un ID numérico único, un dato ideal para aprovechar la propiedad de orden de un BST (todo lo menor al nodo va a la izquierda, todo lo mayor o igual va a la derecha). Esto permite que `buscarPorID()` descarte la mitad del árbol en cada comparación en lugar de revisar todos los gatos uno por uno, algo que una lista (simple, doble o circular) no podría ofrecer sin ordenar y sin una estructura jerárquica.

- `insertar()` / `insertarRecursivo()`: compara el ID del nuevo gato contra cada nodo visitado; si es menor desciende a la izquierda, si es mayor o igual desciende a la derecha, hasta encontrar una posición libre.
- `buscarPorID()`: aprovecha la propiedad de orden del BST, acercándose a una complejidad $O(\log n)$ cuando el árbol está razonablemente balanceado, frente a $O(n)$ de un recorrido lineal.
- `buscarPorNombre()`: al no existir un criterio de orden por nombre, no se puede aplicar la misma optimización, por lo que se realiza un recorrido completo del árbol (izquierda y derecha) — $O(n)$. Esto se explica en el documento para ser honesto sobre los límites reales del BST: solo optimiza la búsqueda por la clave usada al insertar (el ID).
- `preorden()` / `imprimirPreorden()`: recorrido Raíz–Izquierda–Derecha usado por `mostrar()` para listar todos los gatos registrados en el sistema.

Nota importante para la sustentación: en `Gestor_Gatos` los gatos también se guardan en un `HashMap` heredado de `GestorBase` (`getElementos()`), que se usa para el acceso directo $O(1)$ por ID durante el registro y la persistencia en archivo. El árbol binario se mantiene en paralelo y se usa específicamente para las operaciones de búsqueda y recorrido (`buscar()` y `mostrar()`), que es donde su propiedad de orden aporta valor real. Esta convivencia de dos estructuras es un punto que conviene explicar con claridad al profesor: el Map resuelve la persistencia y el acceso directo, mientras que el árbol demuestra y aplica el algoritmo de búsqueda en un BST exigido por el curso.

3.8 Resumen de justificación (para la sustentación oral)

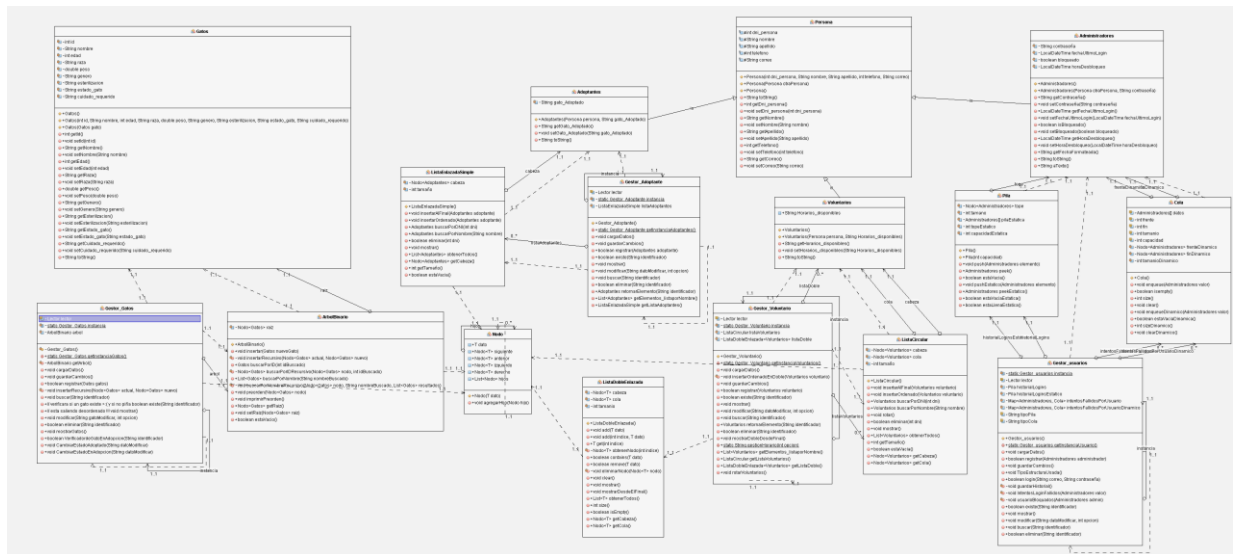
Esta tabla condensa, en una frase, la razón concreta de cada elección — útil como guion rápido al momento de responder la pregunta "¿por qué usaste esta estructura y no otra?":

Estructura	Necesidad real del sistema	Propiedad clave aprovechada
Pila	Saber al instante cuál fue el último login exitoso.	LIFO: el último insertado queda accesible en $O(1)$ con <code>peek()</code> .
Cola	Contar intentos fallidos en orden y bloquear tras N intentos.	FIFO: procesa los intentos en el mismo orden cronológico en que ocurrieron.
Lista Enlazada Simple	Registrar adoptantes sin usar HashMap, con orden alfabético.	Enlace simple hacia adelante: simplicidad y bajo consumo de memoria.
Lista Circular	Rotar el turno de atención entre voluntarios indefinidamente.	El último nodo enlaza con el primero: recorrido cíclico sin lógica extra.
Lista Doble Enlazada	Recorrer voluntarios en reversa y eliminar de forma más flexible.	Cada nodo conoce a su anterior: navegación en ambos sentidos.
Árbol Binario (BST)	Buscar gatos por ID rápidamente entre muchos registros.	Propiedad de orden BST: descarta la mitad del árbol en cada paso.

3.9 Complejidad de las operaciones principales

Estructura	Inserción	Búsqueda	Eliminación
Pila (dinámica)	$O(1)$	$O(1)$ (<code>peek</code>)	—
Cola (dinámica)	$O(1)$	$O(1)$ (<code>frente</code>)	—
Lista Enlazada Simple	$O(n)$ / $O(1)$ en cabeza	$O(n)$	$O(n)$
Lista Doble Enlazada	$O(1)$ en extremos	$O(n)$	$O(1)$ si se tiene el nodo
Lista Circular	$O(n)$	$O(n)$	$O(n)$
Árbol Binario (BST)	$O(\log n)^*$	$O(\log n)^*$	No implementada

4. Diagrama UML



5. Conclusiones

- El proyecto "Animal Shelter" demuestra la aplicación práctica de siete estructuras de datos distintas dentro de un mismo sistema, cada una elegida según las necesidades específicas de la entidad que administra.
- El uso de una clase `Nodo<T>` genérica y reutilizable evidencia un diseño limpio que evita la duplicación de código entre las distintas estructuras.
- La combinación de estructuras estáticas (arreglos) y dinámicas (enlazadas) para Pila y Cola permite comparar directamente sus ventajas y limitaciones de memoria y rendimiento.
- El uso del Árbol Binario de Búsqueda para los gatos, indexado por ID, mejora la eficiencia de búsqueda frente a un recorrido lineal, siempre que el árbol se mantenga razonablemente balanceado.